Structural Bioinformatics with BiochemicalAlgorithms.jl

1st author[1], 2nd author[1], and 3rd author[1]

[1]Scientific Computing and Bioinformatics, Institute of Computer Science, Johannes Gutenberg-University Mainz

ABSTRACT

General Terms

Experimentation, Human Factors

Keywords

Face animation, image-based modelling...

1. Introduction

The aim of structural bioinformatics is the analysis and targeted manipulation of three dimensional structures of biological macromolecules such as proteins and nucleic acids.
Applications include structure minimization, molecular dynamics (MD) simulations and molecular docking scenarios. These molecular modelling techniques typically rely on molecular mechanics in which a molecular force fields is being used to compute the energy of the examining structure. Molecular force fields are just one example of the highly interdisciplinary character of this research field. It combines and applies knowledge from diverse disciplines ranging from fundamental physical laws such as Newton's equation of motion to challenges of numerical computation and to complex situations requiring in-depth knowledge of biochemistry.
Why do we have to run simulations on the computer? Why is it necessary to perform it on a computer? Example Protein Ligand Docking... As an example.
A popular application of molecular simulation is found in the area of drug discovery. Molecular docking aims to find the best configuration for a given protein and a ligand (the drug) and compute the binding affinity. This evident shows the important effect
The first attempts to
Here, we present BiochemicalAlgorithms.jl. We provide the basis for interesting analysis encompassing the entire molecular modelling pipeline:

—Reading common data formats such as PDB, hin, mol and JSON

—Preprocessing the input by preparing the entire system ready to simulate.

—Molecular Mechanics such as AMBER ForceField

—(Output writing) such as JSON

BiochemicalAlgorithms is designed to be a platform from which other packages can be included.

2. BALL - Biochemical Algorithms Library

2.1 Design Goals

2.2 Structure

3. BiochemicalAlgorithms.jl

3.1 Why Julia?

3.2 The core

3.3 The internal representation

When working with structural data of molecular entities such as proteins, or nucleic acid molecules (DNA/RNA), the representation of three dimensional coordinates play an important role. Just like in BALL we define a similar hierarchical structure...as can be seen in fig.. Optimized for random access to any atoms... same hand easy to use for Julia users or Python users familiar with Pandas DataFrames... So initially, representation was based on DataFrames. While theis is extremely intuitive, it has some

3.4 Benchmarks

On par with its C++ predecessor it is intuitiv to write code that is

4. Applications

A trivial comparison between C++ and Julia

4.1 Water molecule

As shown in the code example Code listing 1 the interaction with core components of BiochemicalAlgorithms.jl is straightforward.

Code 1: Intuitive usage of BiochemicalAlgorithms.jl core components

```julia
using BiochemicalAlgorithms
using BiochemicalVisualization

sys = System()
h2o = Molecule(sys)

o1 = Atom(h2o, 1, Elements.O)
h1 = Atom(h2o, 2, Elements.H)
h2 = Atom(h2o, 3, Elements.H)

h1.r = [1, 0, 0]
h2.r = [cos(deg2rad(105)), sin(deg2rad(105)), 0]

Bond(h2o, o1.idx, h1.idx, BondOrder.Single)
Bond(h2o, o1.idx, h2.idx, BondOrder.Single)

```

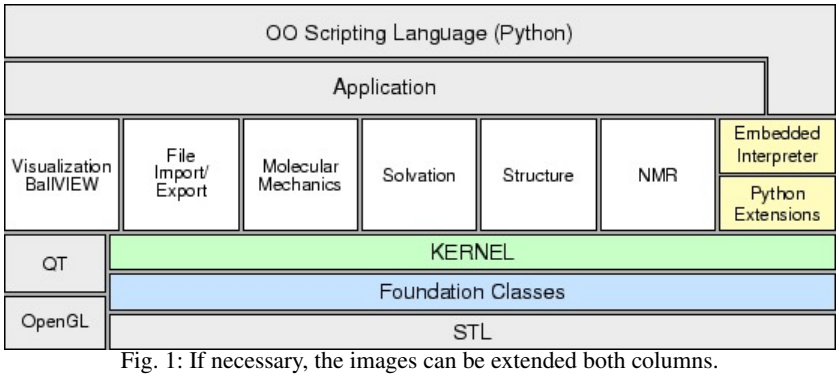

Fig. 1: If necessary, the images can be extended both columns.

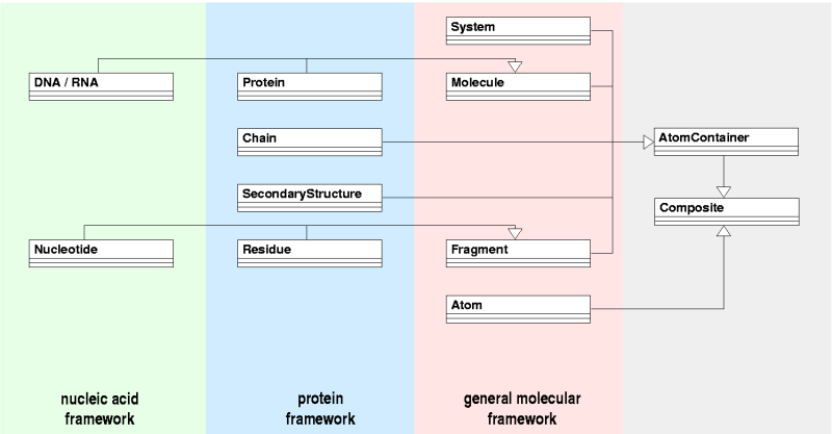

Fig. 2: BALL's architecture is structured in several layers. Upon the standard libary layer are the foundation classes and ontop of them the kernel. Several module extend the interface for visualization, file import and expoert, molecular mechanics, solvation, structure and NMR. The C++ written frameworkis extended by Python interface for fast scripting purposes.

```
17 println("Number of atoms: ", natoms(h2o))
18 println("Number of bonds: ", nbonds(h2o))
19
20 ball_and_stick(sys)
21 stick(sys)
22 van_der_waals(sys)
```

4.2 RMSD computation and Application of Amber

Code 2: Comparison and mapping of two similar structures

```
1  sys = load_pdb.(["data/arnd1.pdb",
2                   "data/arnd2.pdb"])
3
4  fdb = FragmentDB()
5  normalize_names!.(sys, Ref(fdb))
6  reconstruct_fragments!.(sys, Ref(fdb))
7  build_bonds!.(sys, Ref(fdb))
8
9  println.(sys)
10
11 compute_energy.(AmberFF.(sys), verbose=true)
12
13 println("RMSD before mapping: ",
14         compute_rmsd(sys[1], sys[2]))
15
16 map_rigid!(sys[1], sys[2])
17
18 println("RMSD after mapping: ",
19         compute_rmsd(sys[1], sys[2]))
```

4.3 Visualization using BiochemicalVisualization.jl

A key feature of BiochemicalAlgorithms.jl is the visualization tool *BiochemicalVisualization.jl* that has been developed alongside the main framework. As can be seen in Figure 4 BiochemicalVisualization.jl currently supports three different representation of protein structures, namely `ball-and-stick`, `van-der-Waals` and `stick` (cf. code listing 1).

When dealing with three dimensional structures of macromolecules, visualization plays an important role for supporting the development of insights into molecular functions. The possibility to visualize and interactively modify the representations provides great support during modelling scenarios. For instance, the tool has been used to visualize different steps from code listing 2. The first image left represent the raw input read from the underlying pdb file, the image in the middle shows the same input after preprocession it with the fragment data base (lines4-7). Finally, the mapping of both structures is shown in the image on the left. As can be seen, the structures don't match perfectly onto each other.

Even this rather simple example already demonstrates the advantage of a visual representation that can be modified and manipulated in context of modelling scenarios and how the visualization supports the development of knowledge of molecular functions.

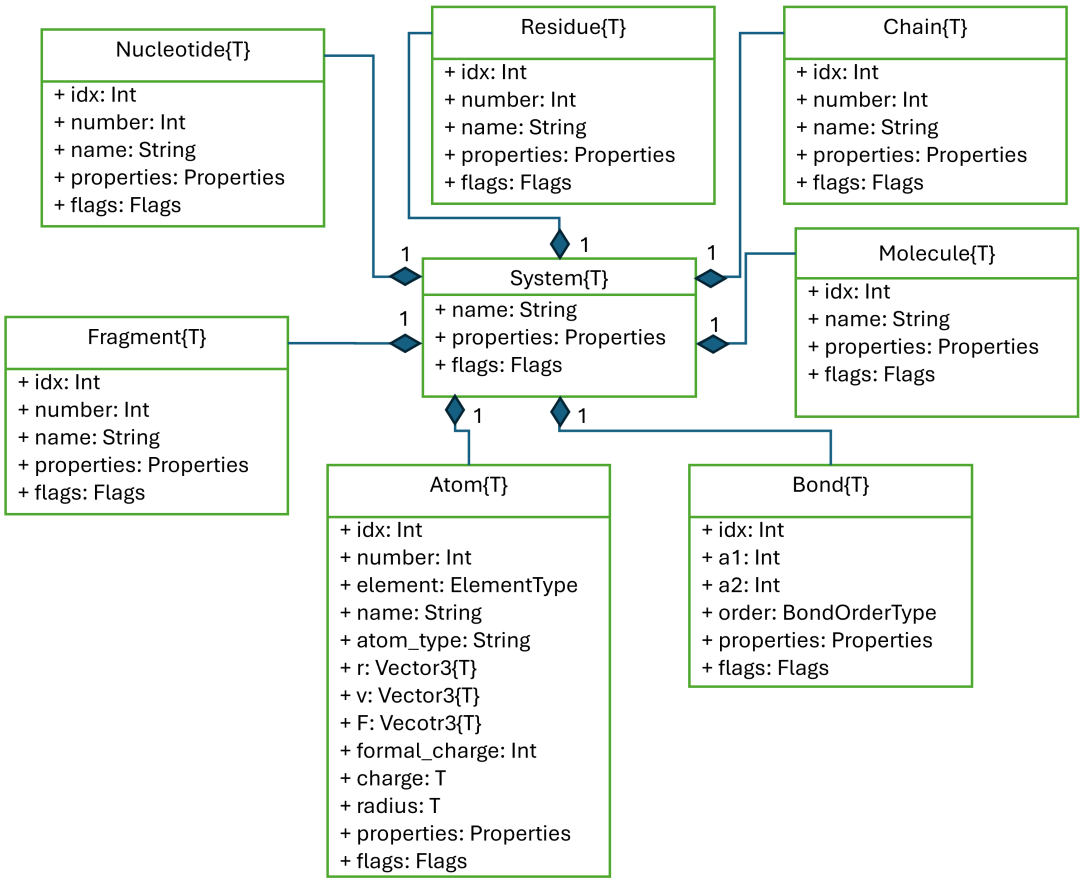

Fig. 3: UML-Diagramm of the core of BiochemicalAlgorithms.jl . In the center resides the `System` interface. All other functionalities are grouped around that core piece. Only the most important functionalities of each class are shown.

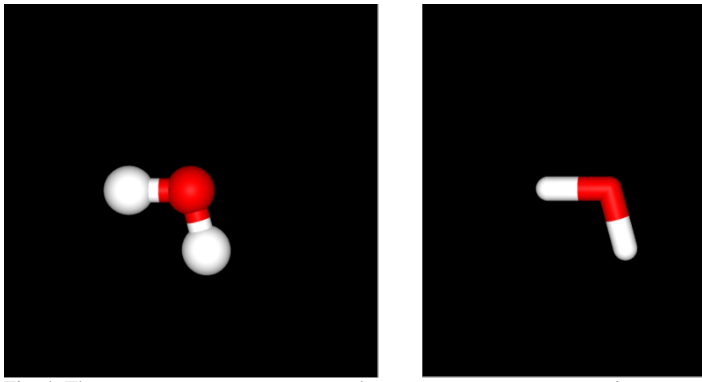

Fig. 4: The `ball-and-stick`, `stick` and `van-der-waals` representation of the code listing 1.

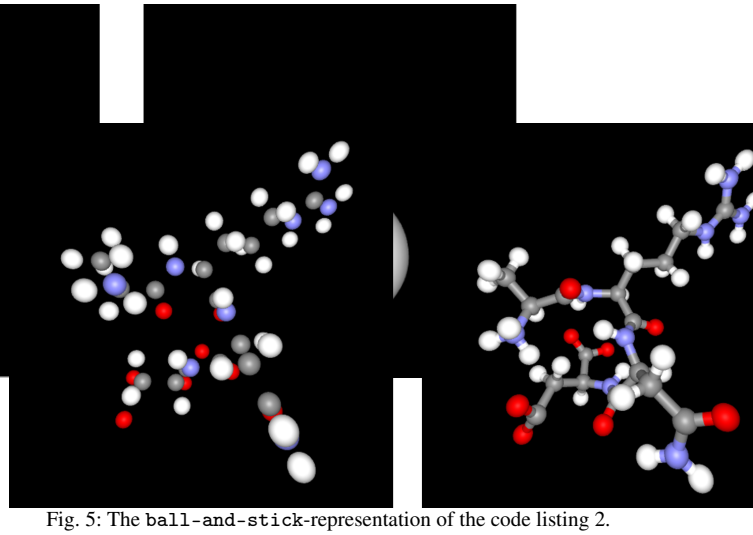

Fig. 5: The `ball-and-stick`-representation of the code listing 2.

5. Integration into Julia Universe

6. Conclusion

7. References